

DATA STRUCTURES AND ALGORITHMS

Unit 3

Linked Lists

*Representation & Memory Allocation · Garbage Collection · Traversal, Search, Insertion & Deletion
Singly, Doubly & Circular Linked Lists · Stacks and Queues using Linked Lists*

What We Will Cover

01

Linked List — Definition & Concept

02

Memory Representation & Allocation

03

Garbage Collection

04

Traversing & Searching

05

Insertion & Deletion — Singly Linked List

06

Doubly Linked List

07

Circular Linked List

08

Stacks using Linked List

09

Queues using Linked List

What is a Linked List?

- A linked list is a linear data structure in which elements (called nodes) are not stored in contiguous memory locations.
- Each node stores two parts:
 - Data — the actual value held by the node
 - Next — a pointer / reference to the address of the next node
- The list is accessed through a special pointer called HEAD, which points to the first node.
- The last node's next part points to NULL, marking the end of the list.
- **Because nodes are linked using pointers rather than fixed positions, the size of a linked list can grow or shrink at run time.**

Structure of a Node



data field | next (address) field

Key Idea

- Arrays need contiguous memory; linked lists don't.
- Insertion/Deletion needs no shifting of elements.
- Memory is allocated dynamically, node by node.

Array vs Linked List

Feature	Array	Linked List
Memory Layout	Contiguous block	Scattered, linked via pointers
Size	Fixed at creation (static)	Grows / shrinks at run time
Access	Direct — $O(1)$ via index	Sequential — $O(n)$, no indexing
Insertion / Deletion	Costly — needs shifting, $O(n)$	Efficient — $O(1)$ if position known
Memory Utilisation	May waste unused allocated space	Uses exactly what is needed, plus pointer overhead
Extra Memory	None required	Extra space needed for pointer/address field

Representation of Linked Lists in Memory

- Nodes are NOT stored in adjacent memory cells — each node can be anywhere in memory (RAM).
- Every node knows the address of the next node, so the list can still be traversed in order.
- The compiler/OS maintains a HEAD pointer holding the address of the first node.
- Logical order (the sequence we perceive) is maintained through pointers, not physical order in memory.
- **In C/C++, a node is typically declared using a self-referential structure:**

```
struct Node {  
    int data;  
    struct Node *next;  
};  
  
struct Node *head = NULL;
```

Logical vs Physical Placement in Memory

1000	data = A next = 1040
1020	(unused / free block)
1040	data = B next = 1080
1060	(unused / free block)
1080	data = C next = NULL

Nodes $A \rightarrow B \rightarrow C$ are logically in order, but physically scattered at addresses 1000, 1040, 1080.

CONCEPT 03

Memory Allocation for Linked Lists

Static Memory Allocation

- Memory is reserved at compile time, before the program runs.
- Used for arrays and fixed-size structures.
- Size cannot change during execution.
- Simple and fast, but wastes memory if unused, or overflows if insufficient.

Dynamic Memory Allocation

- Memory is reserved at run time, from a memory pool called the HEAP.
- A new node is created in memory only at the moment it is needed.
- In C: malloc() / calloc() followed by free()
- In C++: new operator, released using delete
- In Java / Python: handled automatically by the language runtime.
- **This is exactly why a linked list can grow one node at a time.**

CONCEPT 04

Garbage Collection

- When a node is deleted from a linked list, its memory must be released back to the system — otherwise that memory becomes unusable.
- **A memory leak occurs when allocated memory is never freed and is no longer reachable by any pointer.**
- Garbage Collection (GC) is the process of automatically identifying and reclaiming memory occupied by nodes/objects that are no longer referenced by any part of the program.
- **Manual reclamation:**
 - In C, the programmer explicitly calls `free(ptr)`.
 - In C++, the programmer explicitly calls `delete ptr`.
- **Automatic reclamation:**
 - Languages like Java, Python and C# run a background Garbage Collector that tracks reachability and frees unreferenced nodes automatically.
- Common GC technique: Reference Counting — each block keeps a count of how many pointers refer to it; when the count drops to zero, the block is added back to the free list (available list).

Types of Linked Lists

Singly Linked List

Each node points only to the next node.
Traversal is one-directional (forward only).



Doubly Linked List

Each node has two pointers — prev and next — allowing traversal in both directions.



Circular Linked List

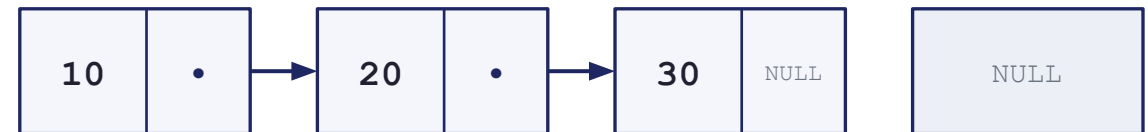
The last node points back to the first node instead of NULL, forming a circle.



Traversing a Linked List

- Traversal means visiting every node of the list exactly once, usually starting from HEAD, to display, count, or process data.
- **Algorithm:**
 - Set a temporary pointer: $ptr = head$
 - While ptr is not NULL:
 - Process / print $ptr \rightarrow data$
 - Move forward: $ptr = ptr \rightarrow next$
 - Stop when ptr becomes NULL (end of list reached).
- **Time Complexity: $O(n)$ — Space Complexity: $O(1)$**

```
void traverse(struct Node *head) {
    struct Node *ptr = head;
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
}
```



ptr starts at HEAD and moves node-by-node until it hits NULL

Searching in a Linked List

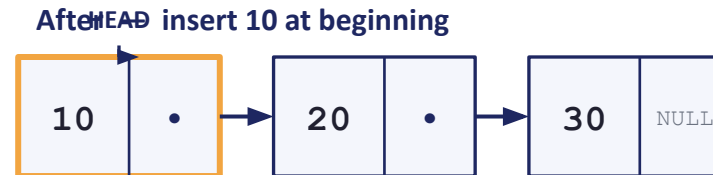
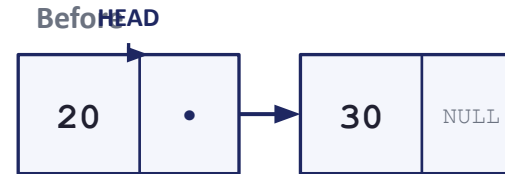
- Searching means finding whether a given key/value is present in the list, and if so, its position.
- **Algorithm:**
 - Set ptr = head, position = 1
 - While ptr is not NULL:
 - If ptr → data equals the key → search successful, return position
 - Else move ahead: ptr = ptr → next, position++
 - If loop ends (ptr == NULL) → element not found.
- Unlike arrays, binary search is not possible directly, because linked lists do not support random access.
- Time Complexity: $O(n)$ in the worst case (linear search)

```
int search(struct Node *head, int key) {  
    struct Node *ptr = head;  
    int pos = 1;  
    while (ptr != NULL) {  
        if (ptr->data == key)  
            return pos;  
        ptr = ptr->next;  
        pos++;  
    }  
    return -1;    // not found  
}
```

Insertion — At the Beginning

- **Steps:**
 - Create a new node and store the value in its data field.
 - Set the new node's next pointer to the current head.
 - Update head to point to the new node.
- **This is the fastest insertion — Time Complexity $O(1)$ — because no traversal is needed.**

```
void insertAtBeginning(int val) {  
    struct Node *n = malloc(sizeof(struct Node));  
    n->data = val;  
    n->next = head;  
    head = n;  
}
```



New node becomes HEAD; its next points to the old first node.

Insertion — At the End & In the Middle

At the End

- Create the new node; set its next to NULL.
- If the list is empty, make it the head.
- Otherwise, traverse using ptr until $\text{ptr} \rightarrow \text{next} == \text{NULL}$ (the last node).
- Link: $\text{ptr} \rightarrow \text{next} = \text{new node}$.
- Time Complexity: $O(n)$ — a full traversal is required to reach the last node ($O(1)$ if a tail pointer is maintained).

In the Middle (after a given node)

- Traverse to the node after which insertion is needed (say, position p).
- Create the new node.
- Set: $\text{new node} \rightarrow \text{next} = \text{ptr} \rightarrow \text{next}$
- Set: $\text{ptr} \rightarrow \text{next} = \text{new node}$
- Order of these two steps matters — the link to the rest of the list must be saved before it is overwritten.
- Time Complexity: $O(n)$ to locate the position, $O(1)$ to link.

Deletion — Beginning, End & Middle

Beginning

```
temp = head
head = head->next
free(temp)
```

End

```
Traverse till the
second-last node (ptr)
free(ptr->next)
ptr->next = NULL
```

Middle (given key)

```
Track prev & curr while
traversing to the node
prev->next = curr->next
free(curr)
```

Important: always free() / delete the removed node's memory to avoid a memory leak (see Garbage Collection).



Doubly Linked List (DLL)

- Each node has **THREE** fields:
 - prev — address of the previous node
 - data — the value stored
 - next — address of the next node
- The prev of the first node and the next of the last node are both NULL.
- **Allows traversal in both directions — forward using next, backward using prev.**
- Needs extra memory per node (one more pointer) compared to a singly linked list.
- Deletion is easier — a node can be removed without needing a separate prev pointer maintained during traversal.

Node Structure



```
struct Node {  
    struct Node *prev;  
    int data;  
    struct Node *next;  
};
```

HEAD → prev is always NULL. TAIL → next is always NULL.

DLL — Insertion & Deletion

Insertion after node p

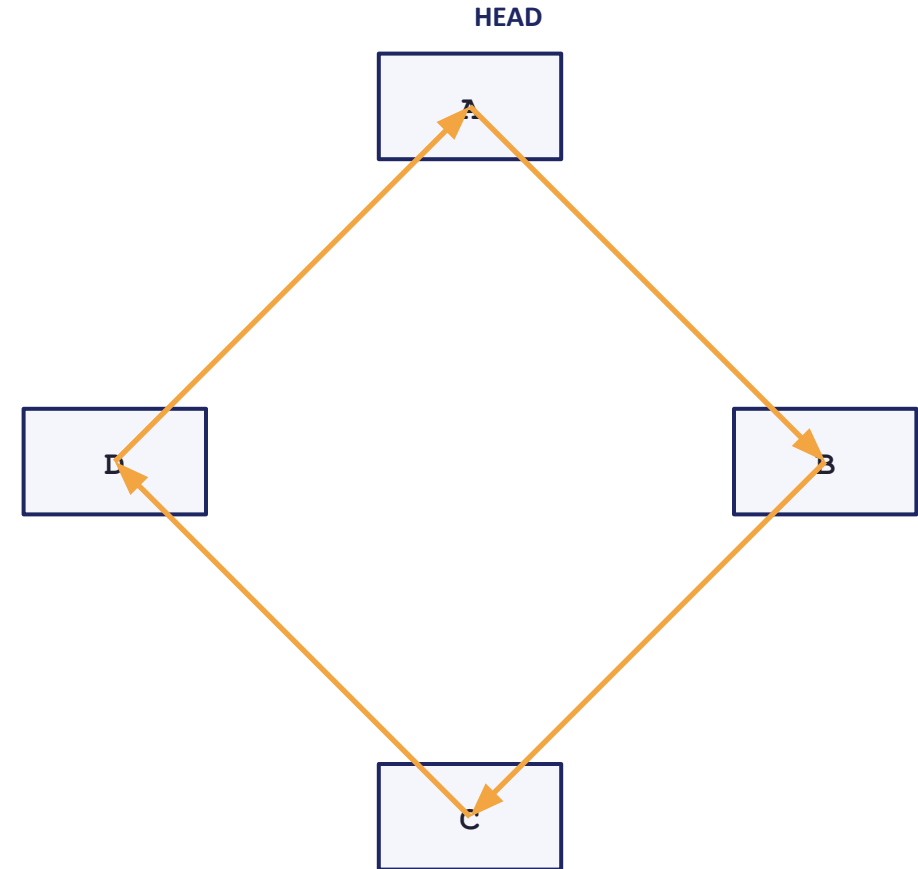
- Create new node n; set n->data = value
- n->next = p->next
- n->prev = p
- If p->next is not NULL: p->next->prev = n
- p->next = n
- Total 4 pointer updates needed (vs 2 in SLL) because both directions must stay consistent.

Deletion of node d

- If d->prev is not NULL: d->prev->next = d->next
- Else: head = d->next (d was the first node)
- If d->next is not NULL: d->next->prev = d->prev
- free(d)
- No need to separately track a 'previous' pointer while traversing — the node itself already knows it via d->prev.

Circular Linked List (CLL)

- In a circular linked list, the last node's next does NOT point to NULL — instead it points back to the first node (head), forming a circle.
- **Two variants:**
 - Singly Circular Linked List — one pointer per node (next), last node → head
 - Doubly Circular Linked List — two pointers per node (prev, next), forms a circle in both directions
- There is no true NULL end — traversal must stop using a condition such as `ptr == head`, not `ptr == NULL`.
- Useful for applications that require cyclic access, e.g. round-robin CPU scheduling, multiplayer turn-based games, circular buffers/playlists.



Last node (D) links back to the first node (A) — no NULL anywhere in the cycle.

CLL — Insertion & Deletion

Insertion (at beginning)

- Create new node n with the given value.
- If list is empty: $n \rightarrow \text{next} = n$; $\text{head} = n$
- Else, traverse to the LAST node (the one whose $\text{next} == \text{head}$).
- $n \rightarrow \text{next} = \text{head}$
- $\text{last} \rightarrow \text{next} = n$
- $\text{head} = n$
- The last node's next must always be re-pointed to the new head.

Deletion (a given node)

- Traverse keeping a trailing pointer prev, until $\text{curr} == \text{node to delete}$.
- If it is the only node: $\text{free}(\text{head})$; $\text{head} = \text{NULL}$
- Else: $\text{prev} \rightarrow \text{next} = \text{curr} \rightarrow \text{next}$
- If curr was head, update: $\text{head} = \text{curr} \rightarrow \text{next}$
- $\text{free}(\text{curr})$
- Always re-check the circular link ($\text{last} \rightarrow \text{next}$) so it still points to the correct head after deletion.

Singly vs Doubly vs Circular — At a Glance

Feature	Singly Linked	Doubly Linked	Circular Linked
Pointers per node	1 (next)	2 (prev, next)	1 or 2
Traversal direction	Forward only	Both directions	Forward (loops back)
Last node points to	NULL	NULL	First node (head)
Memory per node	Lowest	Highest	Same as singly/doubly
Reverse traversal	Not possible directly	Easy (use prev)	Only in doubly-circular
Typical use	Simple lists, stacks	Browser history, LRU cache	Round-robin, buffers

Implementing a Stack using Linked List

- A stack follows LIFO — Last In, First Out.
- The HEAD of the linked list acts as the TOP of the stack.
- **push(x) — insertion always happens at the beginning ($O(1)$):**
 - new node $\rightarrow$ next = top; top = new node
- **pop() — deletion always happens at the beginning ($O(1)$):**
 - temp = top; top = top->next; return temp->data; free(temp)
- Why linked list over array for a stack? No fixed size limit — grows dynamically, no 'stack overflow' from a fixed array bound.

```
void push(int val) {  
    struct Node *n = malloc(sizeof(struct Node));  
    n->data = val;  
    n->next = top;  
    top = n;  
}  
  
int pop() {  
    struct Node *t = top;  
    int val = t->data;  
    top = top->next;  
    free(t);  
    return val;  
}
```



Implementing a Queue using Linked List

- A queue follows FIFO — First In, First Out.
- Two pointers are maintained: FRONT (for deletion) and REAR (for insertion).
- **enqueue(x) — insertion always happens at the REAR end (O(1)):**
 - rear->next = new node; rear = new node
- **dequeue() — deletion always happens at the FRONT end (O(1)):**
 - temp = front; front = front->next; return temp->data; free(temp)
- Why linked list over array for a queue? Avoids the wasted space / shifting problem of array-based queues, and needs no circular-index arithmetic.

```
void enqueue(int val) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = val; n->next = NULL;
    if (rear == NULL) { front = rear = n; return; }
    rear->next = n;
    rear = n;
}

int dequeue() {
    struct Node *t = front;
    int val = t->data;
    front = front->next;
    if (front == NULL) rear = NULL;
    free(t);
    return val;
}
```

Key Takeaways

- A linked list is a dynamic linear structure of nodes connected via pointers — no contiguous memory required.
- Memory for nodes comes from the heap (dynamic allocation); Garbage Collection / `free()`-`delete()` reclaims it.
- Traversal and Searching both run in $O(n)$ — no random access like arrays.
- Insertion/Deletion at a known position is $O(1)$; locating that position is $O(n)$.
- Doubly Linked Lists trade extra memory for two-directional traversal and simpler deletion.
- Circular Linked Lists remove the NULL end, enabling cyclic traversal — useful for scheduling and buffering.
- Stacks (LIFO) and Queues (FIFO) can both be implemented on top of a linked list, giving them dynamic, overflow-free sizing.

Thank You

Questions & Discussion

Unit 3 — Linked Lists | Data Structures and Algorithms
Department of Information & Communication Technology, Parul University